

DevOps

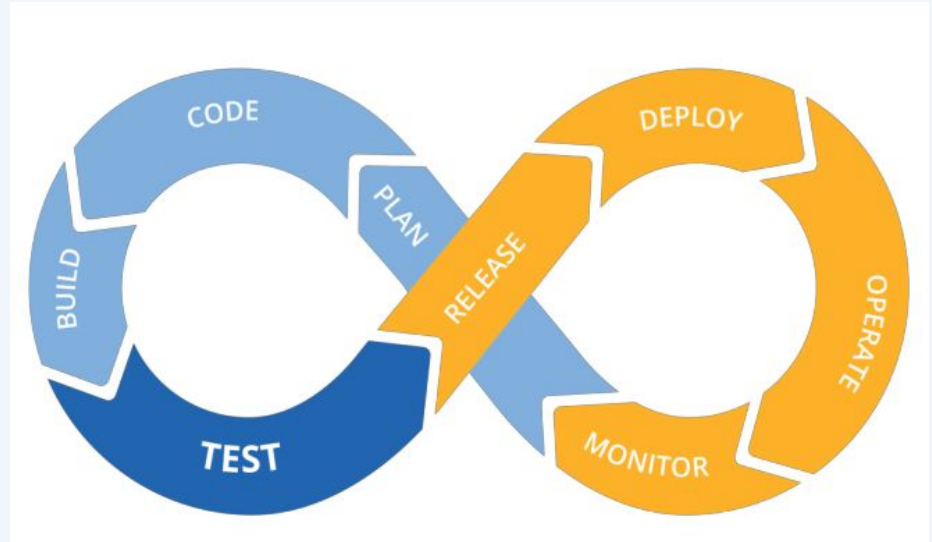
DevOps

DevOps — это методология разработки, которая помогает наладить эффективное взаимодействие разработчиков с другими IT-специалистами. Это набор процессов и инструментов, которые позволяют компании создавать и улучшать продукты быстрее, чем при использовании традиционных подходов к разработке программного обеспечения.

Термин DevOps — это комбинация слов «разработка» (development) и «эксплуатация» (operations), которая отражает процесс интеграции этих дисциплин в единый непрерывный процесс. Разработчики и тестировщики отвечают за Development, а администраторы — за Operations.

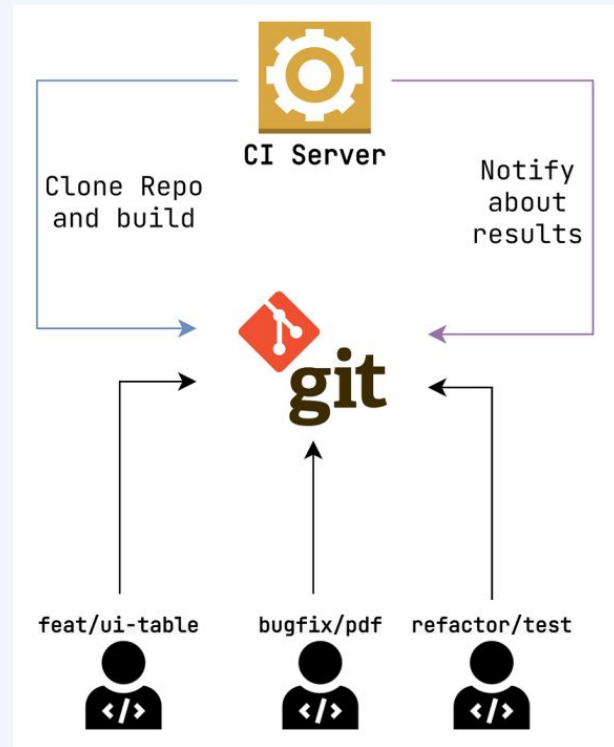
Жизненный цикл DevOps

1. Непрерывное развитие
2. Непрерывная интеграция
3. Непрерывное тестирование
4. Непрерывное развертывание
5. Непрерывный мониторинг
6. Постоянная обратная связь



CI

Непрерывная интеграция (Continuous Integration) — практика разработки программного обеспечения, которая заключается в постоянном слиянии рабочих копий в общую основную ветвь разработки (до нескольких раз в день) и выполнении частых автоматизированных сборок проекта для скорейшего выявления потенциальных дефектов и решения интеграционных проблем. В проекте, где над разными частями системы разработчики трудятся независимо, стадия интеграции является заключительной. Она может непредсказуемо задержать окончание работ. Переход к непрерывной интеграции позволяет снизить трудоемкость интеграции и сделать её более предсказуемой за счет наиболее раннего обнаружения и устранения ошибок и противоречий, но основным преимуществом является сокращение стоимости исправления дефекта за счет раннего его исправления.

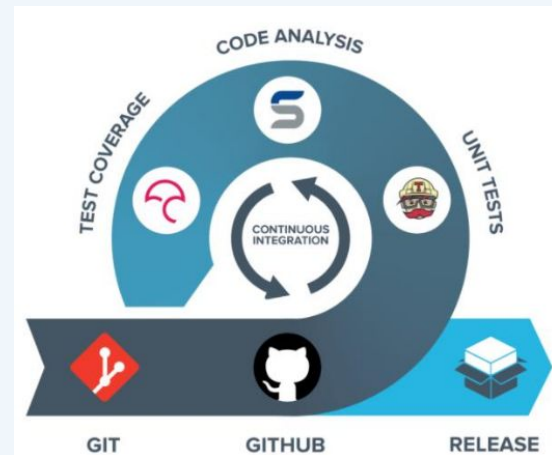


CD

Непрерывная доставка (Continuous delivery) — это подход к разработке программного обеспечения, при котором программное обеспечение производится короткими итерациями, гарантируя, что ПО является стабильным и может быть передано в эксплуатацию в любое время, а передача его не происходит вручную. Целью является сборка, тестирование и релиз программного обеспечения с большей скоростью и частотой. Подход позволяет уменьшить стоимость, время и риски внесения изменений путем более частых мелких обновлений в продакшн-приложение.

СТ

Непрерывное тестирование - это процесс выполнения автоматизированных тестов в рамках конвейера поставки программного обеспечения для получения немедленной обратной связи о бизнес-рисках, связанных с кандидатом на выпуск программного обеспечения. Непрерывное тестирование первоначально было предложено как способ сокращения времени ожидания обратной связи с разработчиками путем введения тестов, запускаемых средой разработки, а также более традиционных тестов, запускаемых разработчиком / тестировщиком..



СТ

В CI-фазе важно, чтобы сборка проходила быстро, поэтому чаще всего применяют облегченные типы тестов:

- **unit-тесты** — чтобы протестировать отдельные компоненты;
- **интеграционные тесты** — чтобы проверить интеграцию этих компонентов;
- **code-style** — чтобы проверить код на соответствие тому, как мы договорились писать его внутри команды;
- **статистический код-анализ** — чтобы отловить потенциальные уязвимости в коде;
- **smoke-тесты** — чтобы проверить, развернулось ли приложение, и корректно ли работают его базовые компоненты.

СТ

На стадии CD идёт непрерывное тестирование конкретных окружений, поэтому тестов становится больше, и они дольше по времени:

- **E2E-тесты** — чтобы проверить корректность работы UI;
- **performance-тесты** — чтобы проверить, нет ли просадок по производительности;
- **регрессионные тесты** — чтобы убедиться, что мы не вернулись к багам, исправленным в прошлом, перед выходом релиза на продакшн-среды;
- **security-тесты**, «ломающие» наше приложение — чтобы проверить, что у нас нет уязвимостей.

Инструменты

- Системы управления версиями (Version Control Systems, VCS)
- Инструменты для автоматической сборки и тестирования (Build and Test Automation Tools)
- Контейнеризация (Containerization)
- Оркестрация контейнеров (Container Orchestration)
- Облачные платформы (Cloud Platforms)

Инструменты для автоматической сборки и тестирования

- 1. Jenkins** - это один из самых популярных инструментов для непрерывной интеграции и непрерывной доставки (CI/CD). Jenkins предоставляет широкий набор плагинов и инструментов, которые позволяют автоматизировать процесс сборки, тестирования и развертывания приложений.
- 2. Travis CI** - это облачный инструмент для непрерывной интеграции и непрерывной доставки. Он позволяет автоматически собирать и тестировать код на каждый коммит в репозиторий, а также интегрировать его с различными платформами и инструментами для непрерывной доставки.
- 3. Circle CI** - это облачный инструмент для автоматической сборки и тестирования. Он позволяет настроить непрерывную интеграцию и непрерывную доставку, а также интегрировать свои приложения с различными платформами облачных вычислений, такими как AWS, GCP и др.
- 4. GitLab CI/CD** - это инструмент для непрерывной интеграции и непрерывной доставки, который полностью интегрирован в GitLab, систему управления версиями. GitLab CI/CD позволяет автоматически собирать и тестировать код, а также развертывать его на различных платформах.
- 5. Selenium** - это инструмент для автоматизированного тестирования веб-приложений. Он позволяет создавать тесты, которые могут эмулировать действия пользователя на веб-страницах и проверять работоспособность приложений.



Инструменты сборки

- Maven
- Gradle
- Ant
- Bazel



Инструмент сборки позволяют:

- подтянуть необходимые библиотеки (в терминах инструментов для сборки — зависимостей (dependency));
- определить как собирать проект и во что (например, собирать в WAR или JAR или executable JAR);
- задать версию проекта в одном месте так, чтобы при сборке была указана она;
- описать проект и его жизненный цикл;
- добавлять так называется плагины (plugin);
- публиковать библиотеки в общем хранилище, чтобы другие проекты смогли подтянуть их как зависимости

Maven фазы сборки

- **validate** - проверка, корректен ли проект и предоставлена ли вся необходимая для сборки информация.
- **compile** - Maven компилирует исходники проекта.
- **test** - проверка скомпилированных файлов.
- **package** - по умолчанию осуществляется в формате JAR. Этот параметр можно изменить, добавив в project тег packaging.
- **integration-test** - Maven обрабатывает и при необходимости распаковывает пакет в среду, где будут выполняться интеграционные тесты.
- **verify** - артефакт проверяется на соответствие критериям качества.
- **install** - артефакт попадает в локальный репозиторий. Теперь его можно использовать в качестве зависимости.
- размещение проекта в удалённом репозитории — **deploy** — финальная стадия работы.

Gradle

Gradle - это инструмент автоматизации сборки проектов, который позволяет разработчикам создавать и управлять проектами на различных платформах и языках программирования. Gradle используется для автоматизации процесса сборки, тестирования, развертывания и управления зависимостями проекта. Gradle использует Groovy и Kotlin DSL (Domain-Specific Language) для описания конфигурации проекта. Как и Maven, Gradle использует файлы конфигурации, но они описываются на Groovy или Kotlin DSL, что делает конфигурацию более гибкой и удобной для разработчиков. С помощью Gradle можно автоматически скачивать и устанавливать зависимости, компилировать и тестировать код, создавать и управлять артефактами (JAR, WAR, EAR), а также выполнять различные другие задачи, связанные с управлением жизненным циклом проекта.

Gradle жизненный цикл

- **Initialization** - Gradle читает и обрабатывает файлы настроек проекта, такие как build.gradle, settings.gradle и т. д. в папке проекта. Затем он настраивает все параметры и связи между проектами, подпроектами, плагинами и зависимостями.
- **Configuration** - Gradle определяет список задач, которые должны быть выполнены для сборки проекта на основе файла настроек. Он определяет порядок выполнения задач, зависимости между задачами и проверяет корректность настроек.
- **Execution** - Gradle выполняет задачи, начиная с задач, необходимых для сборки приложения, и заканчивая зависимыми задачами. Он также следит за тем, чтобы каждая задача выполнялась только один раз.
- **Completion** - Gradle завершает выполнение всех задач и выводит отчет об успешном или неуспешном выполнении.

Виртуализация

Виртуализация — предоставление набора вычислительных ресурсов или их логического объединения, абстрагированное от аппаратной реализации, и обеспечивающее при этом логическую изоляцию друг от друга вычислительных процессов, выполняемых на одном физическом ресурсе. Примером использования виртуализации является возможность запуска нескольких операционных систем на одном компьютере: при том каждый из экземпляров таких гостевых операционных систем работает со своим набором логических ресурсов (процессорных, оперативной памяти, устройств хранения), предоставлением которых из общего пула, доступного на уровне оборудования, управляет хостовая операционная система — гипервизор. Также могут быть подвергнуты виртуализации сети передачи данных, сети хранения данных, платформенное и прикладное программное обеспечение.

Основные виды виртуализации

Аппаратная виртуализация - позволяет создавать независимые и изолированные друг от друга виртуальные компьютеры с помощью программной имитации ресурсов (процессора, памяти, сети, диска и др.) физического сервера. Физический сервер называют хостовой машиной (хостом), виртуальные компьютеры – виртуальными машинами, VM (их также называют гостями). Программное обеспечение, которое создает виртуальные машины и управляет ими, называют гипервизором (а также виртуальным монитором или контрольной программой). На практике на виртуальных машинах могут использоваться разные ОС для разных целей – например, Windows Server под контроллер домена Active Directory и Debian под веб-сервер NGINX.

Виртуализация рабочих столов – позволяет отделить логический рабочий стол (набор пользовательских программ, работающий под ОС) от физической инфраструктуры (например, персональных компьютеров). Одной из наиболее распространенных форм виртуализации рабочих столов является VDI (Virtual Desktop Infrastructure) – инфраструктура виртуальных рабочих столов. Каждый пользователь VDI имеет программную имитацию ОС с необходимым набором программ на физическом сервере под управлением гипервизора и может подключаться к ней по сети. На практике VDI может использоваться для работы большого количества сотрудников на «удаленке» для того, чтобы не закупать им отдельные рабочие станции.

Виртуализация на уровне ОС (контейнеризация) - позволяет запускать программное обеспечение в изолированных на уровне операционной системы пространствах. Наиболее распространенной формой виртуализации на уровне ОС являются контейнеры (например, Docker). Контейнеры более легковесны, чем виртуальные машины, так как они опираются на функционал ядра ОС и им не требуется взаимодействовать с аппаратным обеспечением. На практике контейнеры представляют из себя изолированную среду для запуска любого приложения со всеми его зависимостями и настройками.

Виртуализация серверов

Виртуализация серверов – это процесс разделения физического сервера на несколько уникальных и изолированных виртуальных машин (серверов) с помощью программного обеспечения (гипервизора). На каждом виртуальном сервере могут независимо выполняться собственные операционные системы.

Виртуализация серверов позволяет:

- Оптимизировать затраты на покупку серверного оборудования. Под каждую задачу выделяется виртуальный сервер с необходимым количеством ресурсов (ЦПУ, ОЗУ и др.), простои оборудования минимизируются.
- Упростить сопровождение инфраструктуры. Создание, удаление или обслуживание виртуальной машины как правило проще и быстрее, чем аналогичные операции с физическим сервером.
- Повысить отказоустойчивость инфраструктуры. Виртуальные машины изолированы друг от друга, программный сбой на одной из них не приведет к потере работоспособности сервисов и приложений на остальных.

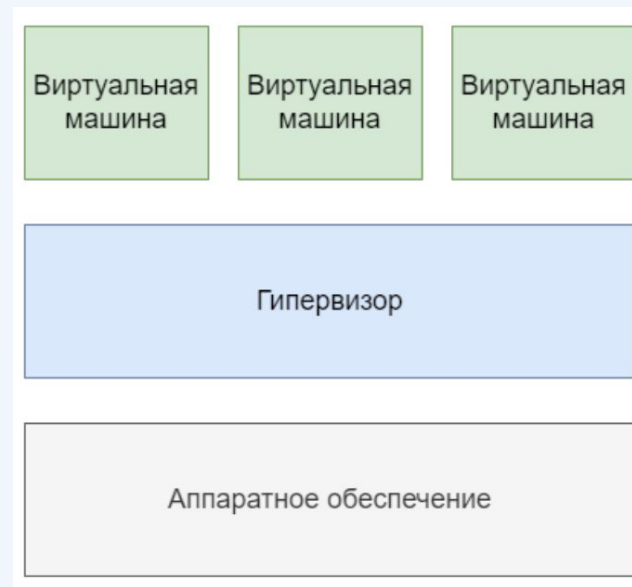
Гипервизоры первого типа

Гипервизор первого типа выполняется как контрольная программа непосредственно на аппаратной части компьютера и не требует ОС общего назначения. В данной архитектуре гипервизор управляет распределением вычислительных ресурсов и сам контролирует все обращения виртуальных машин к устройствам.

Гипервизоры первого типа показывают высокое быстродействие, однако обладают очевидным недостатком – необходимость поддерживать драйверы устройств приводит к сужению списка совместимого аппаратного обеспечения.

Примеры:

- VMWare ESXi;
- KVM (Proxmox VE) – может быть отнесен ко второму типу;
- Xen (Xenserver, Citrix Hypervisor)
- Hyper-V – могут быть также отнесены к гибриднему типу.



Гипервизоры второго типа

Гипервизор второго типа выполняется поверх хостовой операционной системы (как правило Linux). Он управляет гостевыми операционными системами, в то время как эмуляцией и управлением физическими ресурсами занимается хостовая ОС.

Гипервизоры второго типа показывают меньшее относительно гипервизоров первого типа быстродействие и реже используются в промышленной эксплуатации, однако отлично подходят для задач обучения и разработки ПО.

Примеры:

- Oracle VM VirtualBox, VMWare Workstation;
- KVM (Proxmox VE) – может быть также отнесен к первому типу.



Гипервизоры гибридного типа

Гибридный гипервизор сочетает в себе характеристики гипервизоров первого и второго типов – он выполняется поверх специализированной сервисной (или базовой) операционной системы. Сервисная ОС называется родительским разделом или доменом (parent partition в терминологии Hyper-V или domain dom0 в терминологии Xen). После установки гипервизора ядро ОС переходит в режим поддержки виртуализации и передает управление ресурсами процессора и памяти гипервизору. При этом родительский раздел берет на себя функцию обработки обращений к драйверам устройств и операциям ввода-вывода. Данный подход удобен с точки зрения совместимости с оборудованием: не требуется добавлять в гипервизор драйверы устройств (расширяется список совместимого аппаратного обеспечения). Также гипервизор освобождается от задачи обработки вызовов к драйверам устройств – эти вызовы обрабатывает сервисная ОС.

Примеры:

- Xen (XenServer, Citrix Hypervisor),
- Hyper-V – могут быть также отнесены к первому типу



Контейнеризация

Контейнеризация (виртуализация на уровне ОС) – технология, которая позволяет запускать программное обеспечение в изолированных на уровне операционной системы пространствах. Контейнеры являются наиболее распространенной формой виртуализации на уровне ОС. С помощью контейнеров можно запустить несколько приложений на одном сервере (хостовой машине), изолируя их друг от друга.



Механизмы изоляции контейнеров

Изоляция процессов в контейнерах осуществляется благодаря двум механизмам ядра Linux – пространствам имен (namespaces) и контрольным группам (cgroups). Пространства имен гарантируют, что процесс будет работать с собственным представлением системы. Существует несколько типов пространств имен:

- файловая система (mount, mnt) – изолирует файловую систему
- UTS (UNIX Time-Sharing, uts) – изолирует хостнейм и доменное имя
- идентификатор процессов (process identifier, pid) – изолирует процессы
- сеть (network, net) – изолирует сетевые интерфейсы
- межпроцессное взаимодействие (ipc) – изолирует конкурирующее взаимодействие процессами
- пользовательские идентификаторы (user) – изолирует ID пользователей и групп
-

Процесс принадлежит не одному пространству имен, а одному пространству имен каждого типа. Контрольные группы гарантируют, что процесс не будет конкурировать за ресурсы, зарезервированные за другими процессами. Они ограничивают (контролируют) объем ресурсов, который процесс может потреблять – ЦПУ, ОЗУ, пропускную способность сети и др.

Основные понятия

Container image (образ) – файл, в который упаковано приложение и его среда. Он содержит файловую систему, которая будет доступна приложению, и другие метаданные (например команды, которые должны быть выполнены при запуске контейнера). Образы контейнеров состоят из слоев (как правило один слой – одна инструкция). Разные образы могут содержать одни и те же слои, поскольку каждый слой надстроен поверх другого образа, а два разных образа могут использовать один и тот же родительский образ в качестве основы. Образы хранятся в Registry Server (реестре) и версионизируются с помощью tag (тегов). Если тег не указан, то по умолчанию используется latest. Примеры: Ubuntu, Postgres, NGINX.

Registry Server (реестр, хранилище) – это репозиторий, в котором хранятся образы. После создания образа на локальном компьютере его можно отправить (push) в хранилище, а затем извлечь (pull) на другом компьютере и запустить его там. Существуют общедоступные и закрытые реестры образов. Примеры: Docker Hub (репозитории docker.io), RedHat Quay.io (репозитории quay.io).

Основные понятия

Container (контейнер) – это экземпляр образа контейнера. Выполняемый контейнер – это запущенный процесс, изолированный от других процессов на сервере и ограниченный выделенным объемом ресурсов (ЦПУ, ОЗУ, диска и др.). Выполняемый контейнер сохраняет все слои образа с доступом на чтение и формирует сверху свой исполняемый слой с доступом на запись.

Container Engine (движок контейнеризации) – это программная платформа для упаковки, распространения и выполнения приложений, которая скачивает образы и с пользовательской точки зрения запускает контейнеры (на самом деле за создание и запуск контейнеров отвечает Container Runtime). Примеры: Docker, Podman.

Container Runtime (среда выполнения контейнеров) – программный компонент для создания и запуска контейнеров. Примеры: runc (инструмент командной строки, основанный на упоминавшейся выше библиотеке libcontainer), crun.

Host (хост) – сервер, на котором запущен Container Engine и выполняются контейнеры.

Docker

Docker – это открытая платформа для разработки, доставки и запуска приложений. Состоит из утилиты командной строки `docker`, которая вызывает одноименный сервис (сервис является потенциальной единой точкой отказа) и требует права доступа `root`. По умолчанию использует в качестве Container Runtime `runc`. Все файлы Docker (образы, контейнеры и др.) по умолчанию хранятся в каталоге `/var/lib/docker`.

Docker

При запуске контейнер получает доступ на чтение ко всем слоям образа, а также создает свой исполняемый слой с возможностью создавать, обновлять и удалять файлы. Все эти изменения не будут видны для файловой системы хоста и других контейнеров, даже если они используют тот же базовый образ. При удалении контейнера все измененные данные также будут удалены. В большинстве случаев это предпочтительное поведение, однако иногда данные необходимо расшарить между несколькими контейнерами или просто сохранить.

Рассмотрим два способа хранения данных контейнеров:

named volumes – именованные тома хранения данных. Позволяет сохранять данные в именованный том, который располагается в каталоге в `/var/lib/docker/volumes` и не удаляется при удалении контейнера. Том может быть подключен к нескольким контейнерам

bind mount – монтирование каталога с хоста. Позволяет монтировать файл или каталог с хоста в контейнер. На практике используется для проброса конфигурационных файлов или каталога БД внутрь контейнера

Docker Compose

Docker Compose – это инструмент для декларативного описания и запуска приложений, состоящих из нескольких контейнеров. Он использует `yaml` файл для настройки сервисов приложения и выполняет процесс создания и запуска всех контейнеров с помощью одной команды. Утилита `docker-compose` позволяет выполнять команды на нескольких контейнерах одновременно – создавать образы, масштабировать контейнеры, запускать остановленные контейнеры и др.

Оркестрация контейнеров

- это процесс управления и автоматизации запуска, масштабирования и управления жизненным циклом контейнеризованных приложений в кластере или виртуальной среде. Это позволяет разработчикам упростить и автоматизировать процесс развертывания приложений, обеспечить высокую доступность и масштабируемость, а также управлять инфраструктурой как кодом.

Задачи оркестраторов

Подготовка и развертывание инфраструктуры. Включает в себя установку приложений (и их зависимостей) на заранее подготовленный (снабженный нужными службами, библиотеками и т.п.) сервер.

Планирование и конфигурация. Здесь имеется в виду настройка процессов, происходящих на сервере, при помощи планировщиков. Подход справедлив как для монолитов, так и для микросервисов. Удобство заключается в том, что системы оркестрации заранее включают в себя необходимый набор программ-планировщиков, поэтому надобность в их ручной установке отпадает.

Распределение ресурсов. Деплой сервиса или приложения предусматривает выделение серверных ресурсов, таких как RAM и процессорное время. Оркестратор позволяет установить пороговые и нормальные параметры работы и поддерживать приложения «в строю» с минимальными «накладными» затратами ресурсов.

Масштабирование контейнеров согласно рабочим нагрузкам. Здесь дополнительные комментарии излишни: слишком большое выделение ресурсов невыгодно, слишком малое способно вызвать задержки и сбои в работе приложения. «Умное» масштабирование на основе реальной нагрузки позволяет свести неэффективные траты к минимуму, сохранив работоспособность приложения на должном уровне.

Задачи оркестраторов

Балансировка нагрузки. Балансировщики позволяют распределить нагрузку на серверную часть и находящиеся на ней контейнеры. Это сокращает потребление мощностей, нормализует пропускную способность на канале и повышает скорость отклика.

Маршрутизация. Под маршрутизацией понимается настройка распределения трафика из глобального интернета по внутренним ресурсам приложения.

Отслеживание состояния контейнеров. С помощью мониторинга можно в реальном времени отслеживать контейнеры и их параметры: когда, где и какие их образы были запущены, какие внутри контейнеров используются команды, где завышено потребление ресурсов и т.п.

Обеспечение безопасности. Система выдает администратору предупреждения, основанные на наблюдениях за кластерами: данные о найденных уязвимостях и проблемах конфигурации, а также советы по устранению обнаруженных проблем.

Kubernetes

Эта портативная платформа с открытым исходным кодом считается отраслевым стандартом. Поддерживает и декларативную конфигурацию объектов, и автоматизацию. Можно автоматизировать масштабирование, развертывание с помощью шаблонов и управление рабочей нагрузкой и сервисами контейнеров. Поддерживает ОС: Windows, macOS, Linux.

Предлагает:

- мониторинг сервисов и распределение нагрузки,
- автомонтирование гибкой системы хранения,
- автоматизированные откаты и развертывания,
- автораспределение нагрузки,
- горизонтальное автомасштабирование,
- автоперезапуск, замена и завершение работы отказавших контейнеров,
- управление конфигурацией и конфиденциальной информацией,
- самовосстановление.



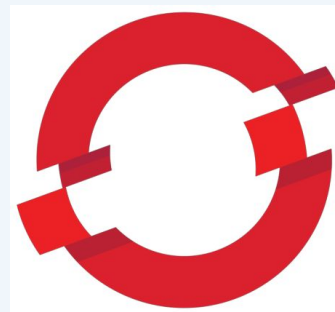
Openshift

Платформа корпоративного уровня в формате PaaS для разработки, развертывания и управления не только контейнерными, но и классическими приложениями и их компонентами в режиме самообслуживания. Позволяет автоматизировать их в физических, виртуальных и общедоступных, частных и гибридных облачных средах.

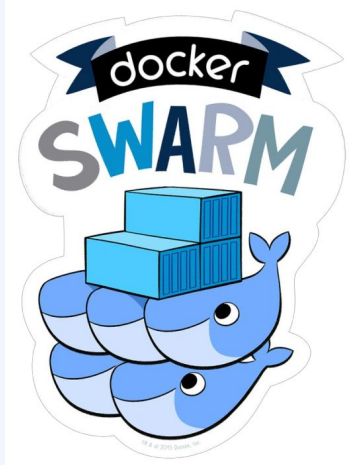
Поддержка ОС: Linux.

Предлагает:

- встроенные средства планирования, кластеризации и оркестрации;
- автомасштабирование;
- балансировку нагрузки;
- по заявлению производителя, «функции безопасности полностью устраняют риски вмешательства Tenant-клиентов в работу других приложений или хоста»;
- возможность подключить постоянное хранилище непосредственно к контейнерам Linux, благодаря чему можно использовать платформу для stateful- (без сохранения данных клиента для следующей сессии) и stateless-приложений (с сохранением данных);
- совместимость созданных на платформе приложений с любыми другими платформами, поддерживающими контейнеры Docker.



другие инструменты



оркестровка контейнеров

Nomand



пакетный менеджер для
настройки, установки и
обновления приложений в
Kubernetes



удаленное управление конфигурациями

Литература

- <https://habr.com/ru/articles/458046/>
- <https://habr.com/ru/articles/657677/>
- <https://habr.com/ru/companies/domclick/articles/566224/>
- <https://habr.com/ru/companies/ruvds/articles/450312/>
- <https://mcs.mail.ru/blog/what-are-container-orchestration-systems>
- <https://habr.com/ru/companies/dataart/articles/588258/>